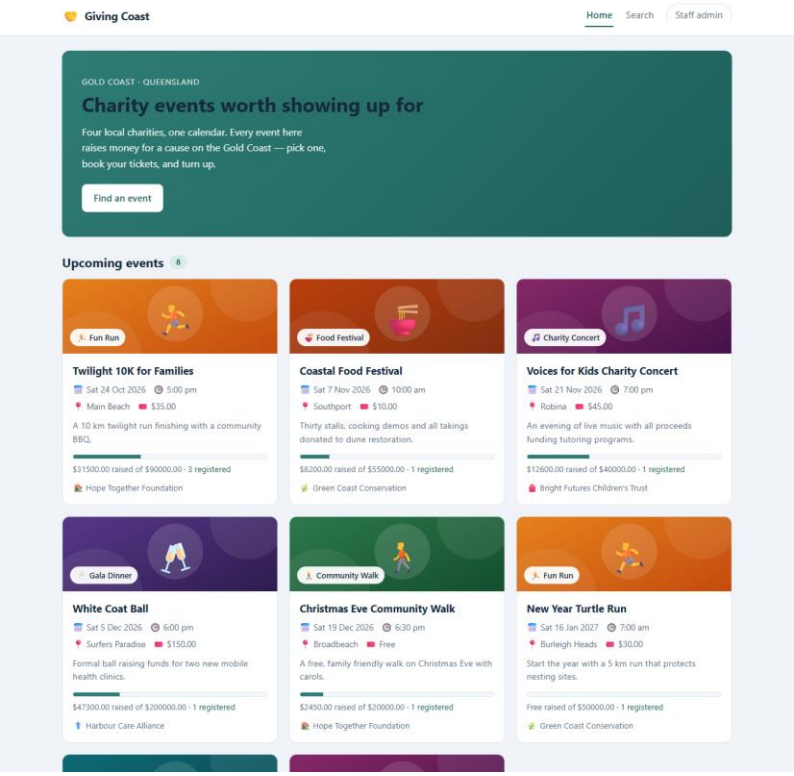
Storyboard

TITLE: Home - Giving Coast public site SCREEN ID: S1 DATE: 18 October 2026

Elements:

GRAPHICS:



Live screenshot of the deployed page (https://24831941.it.scu.edu.au), taken with headless Chrome. The layout is the design; every value on it comes from the API.

TEXT:

Hero: eyebrow 'Gold Coast - Queensland', headline 'Charity events worth showing up for', lead paragraph
Button: 'Find an event'
Section headings: 'Upcoming events' / 'Recently finished'
On every card: event name, date, time, venue, price, one-line summary, '$X raised of $Y goal', 'N registered', charity name

AUDIO/VIDEO:

None. No audio, no video, no images: the icons are characters from the database, the colour bands and progress bar are CSS drawn from API values.

NOTES

NOTES:

The first screen is already the product: a visitor reaches an event in at most one click, and a broken back end shows immediately as an empty grid with a retry button rather than a silent blank page.

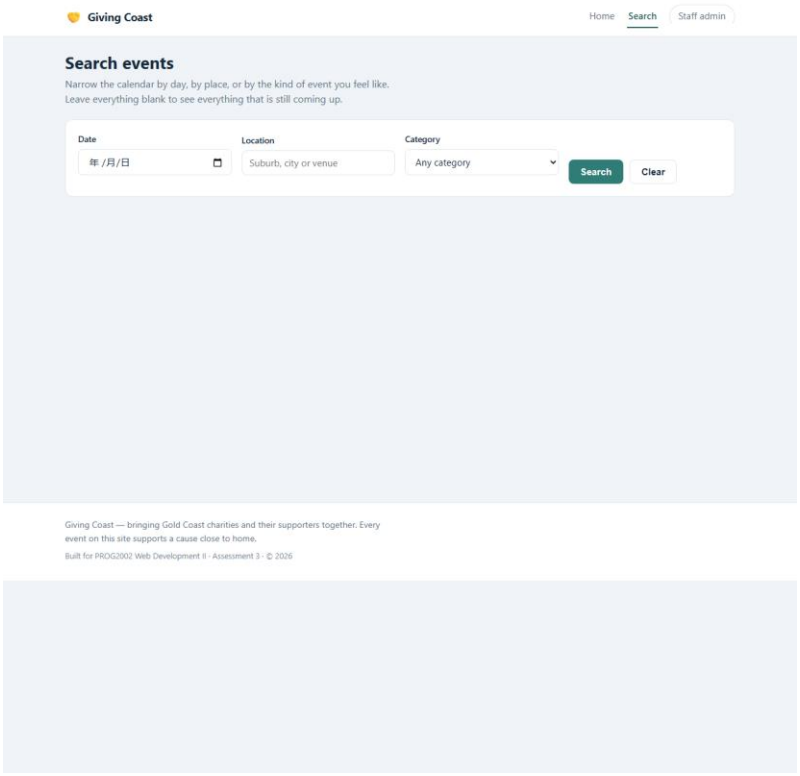
Description:	The landing page. Its job is to make a visitor want to attend something and get them to an event within one click. It also quietly proves the site is alive: every card is rendered from the database, so an empty home page would mean a broken back end.
Content:	Header with the Giving Coast mark and the menu (Home / Search / Staff admin). Hero panel: eyebrow 'Gold Coast - Queensland', headline 'Charity events worth showing up for', a two-line lead, and the 'Find an event' button. 'Upcoming events' - a grid of event cards, each showing the category chip with its icon, the event name, date, time, venue and price, a one-line summary, a progress bar of raised against goal, how many people have registered, and the running charity. 'Recently finished' - the same card grid for past events. Footer.
Interactions:	On load the page calls GET /api/events and GET /api/events/past and renders the two grids from the responses; nothing on the page is hard-coded. 'Find an event' and the menu Search item go to the search page. Clicking any card opens that event's detail page. If the API is unreachable the grids show an error panel with a 'Try again' button instead of a blank page.
Media Creation:	No image assets at all. The category icons are emoji stored in the database and served by the API (which is why the tables are utf8mb4); the card colour bands are CSS derived from the category; the progress bar is drawn from raised_amount / goal_amount returned by the API rather than stored in the table.
Navigation:	In: the site root, or 'View public site' from the staff site. Out: Search (hero button and menu), Event detail (any card), Staff admin (menu).

Storyboard

TITLE: Search - find an event SCREEN ID: S2 DATE: 18 October 2026

Elements:

GRAPHICS:



Live screenshot of the deployed page (https://24831941.it.scu.edu.au), taken with headless Chrome. The layout is the design; every value on it comes from the API.

TEXT:

Title: 'Search events'
Instruction: 'Leave everything blank to see everything that is still coming up.'
Field labels: Date / Location / Category
Buttons: 'Search' / 'Clear'
Results heading with the number found

AUDIO/VIDEO:

None - same card component as the home page; the category icons come from the database.

NOTES

NOTES:

Filtering runs on the server, so the page only draws results; 'no search yet' and 'no results' are deliberately two different states. A past date is refused before the request is sent.

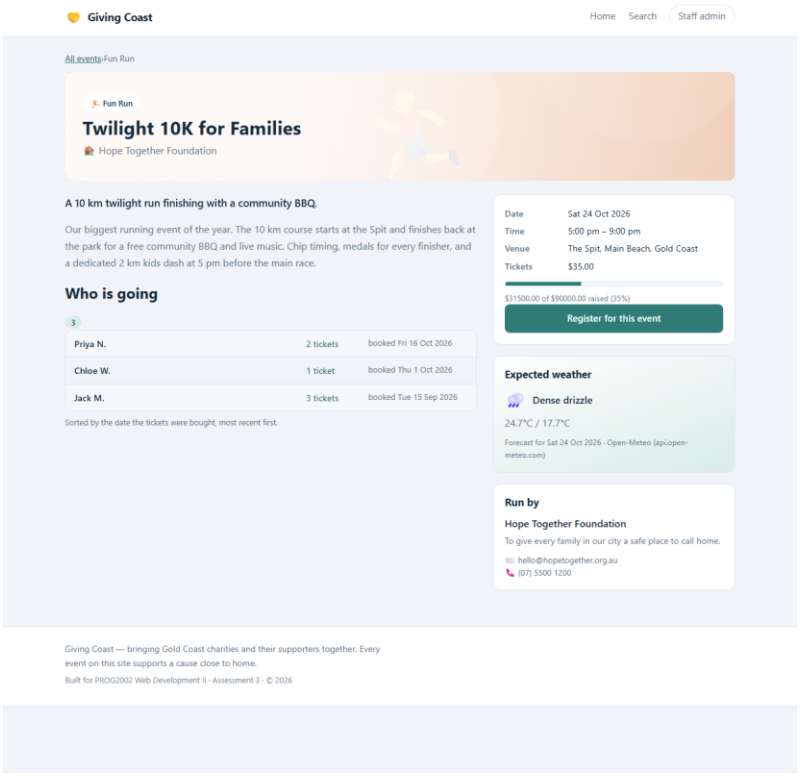
Description:	For the visitor who already knows roughly what they want. Three filters - date, location and category - any combination of which can be used, or none at all.
Content:	Page title and a two-line explanation ('Leave everything blank to see everything that is still coming up'). Filter bar: a date input (minimum = today), a location text input (suburb, city or venue), a category select, and Search / Clear buttons. A results heading with the number found, then the same event cards the home page uses, so the two pages can never look or behave differently.
Interactions:	Search sends GET /api/events?date=&location=&category= - the filtering is done by the API, not by downloading every event and hiding rows in the browser. A past date is rejected before the request is sent, with a message instead of a pointless round trip. The API's validation message is shown verbatim if it rejects a value. Clear empties the filters and forgets the previous results.
Media Creation:	The category select is filled from GET /api/categories, so a new category added by staff appears here without a rebuild. Cards reuse the home page component.
Navigation:	In: 'Find an event' from the home hero, or Search in the menu. Out: any card opens the event detail page; the menu goes home.

Storyboard

TITLE: Event detail - one charity event SCREEN ID: S3 DATE: 18 October 2026

Elements:

GRAPHICS:



Live screenshot of the deployed page (https://24831941.it.scu.edu.au), taken with headless Chrome. The layout is the design; every value on it comes from the API.

TEXT:

Breadcrumb: 'All events › {category}' and a 'Back to all events' link
Panels: 'Who is going' (name, tickets, booking date), 'Expected weather' (icon, description, max/min degrees), 'Run by' (charity details)
Button: 'Register'

AUDIO/VIDEO:

None. The weather icon is a character (emoji) mapped from the Open-Meteo numeric code by our own API, not an image file.

NOTES

NOTES:

Social proof sits next to the price because it is the strongest reason to book; the weather is a separate request so a slow forecast never delays the page.

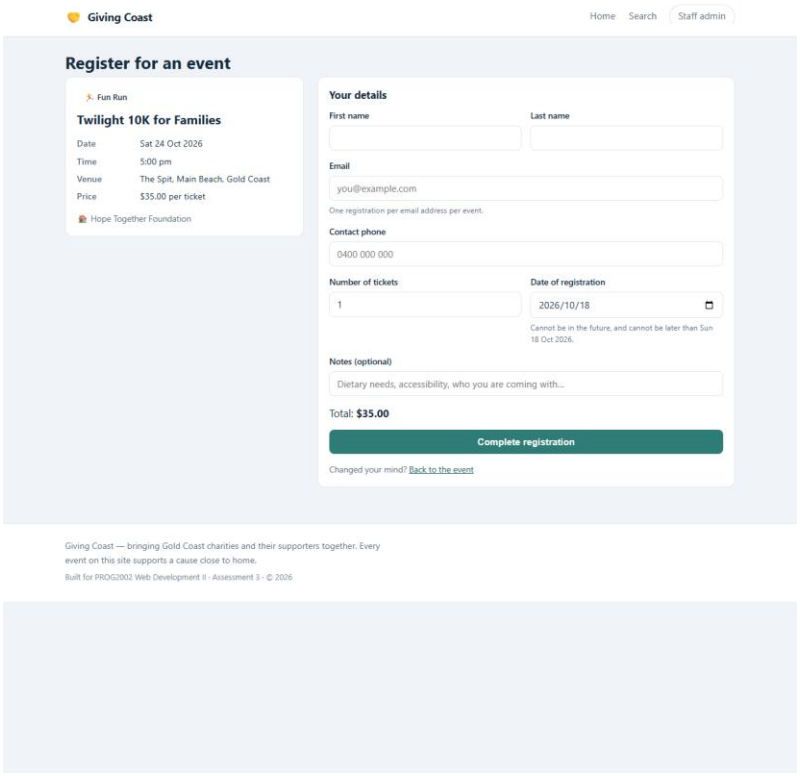
Description:	Everything about one event, and the door to booking it. This is the page a visitor decides on, so it carries the practical details, the social proof and the weather in one view.
Content:	Breadcrumb (All events > category) and a back link. The event: name, date, start and end time, venue and city, ticket price, the full description, the progress bar, and a 'Run by' panel with the charity's details. 'Who is going' - the registration list with each person's name, tickets and booking date, newest first, plus a count. 'Expected weather' panel for the event day. The Register button.
Interactions:	One call to GET /api/events/:id returns the event and its registrations together, so the page is never half-loaded and the list can never disagree with the event. The weather arrives from GET /api/weather/:eventId as a separate request, so a slow or failing forecast never delays the page. Register goes to /register?eventId=... - the event travels in the URL, not in a JavaScript variable, so the page survives a refresh, a bookmark and a shared link.
Media Creation:	The weather panel is fed by our own API, which calls Open-Meteo with the event's latitude and longitude and maps the numeric weather codes to words and an icon. If the upstream service is down the panel says there is no forecast rather than showing an empty box.
Navigation:	In: a card on the home or search page. Out: Register (the button), back to all events (breadcrumb).

Storyboard

TITLE: Register - book a place SCREEN ID: S4 DATE: 18 October 2026

Elements:

GRAPHICS:



Live screenshot of the deployed page (https://24831941.it.scu.edu.au), taken with headless Chrome. The layout is the design; every value on it comes from the API.

TEXT:

Title: 'Register for an event'; left panel shows the event name, date, time, venue and price per ticket
Form heading 'Your details'; labels: First name / Last name / Email / Contact phone / Number of tickets / Date of registration / Notes (optional)
Live 'Total: $X'; button 'Complete registration'
On success the form is replaced by 'You are registered' plus the booking details and a back link

AUDIO/VIDEO:

None - the page is form and data only. The total is computed live from tickets x price.

NOTES

NOTES:

Two columns: what you are booking on the left, who you are on the right - reasons before demands. Each field carries its own message instead of one generic alert, and a 409 shows the API's sentence verbatim.

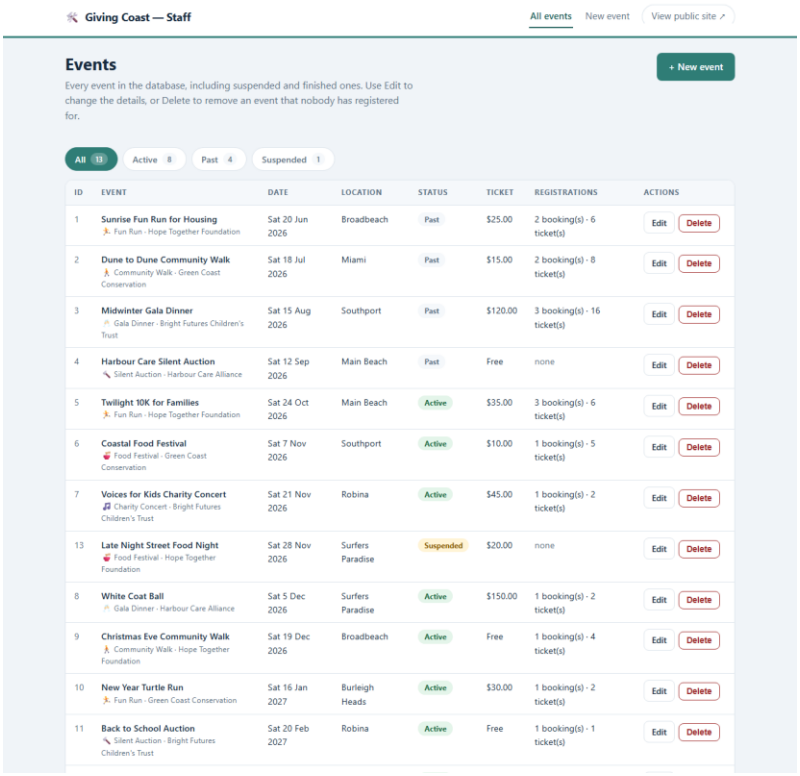
Description:	The one page where we ask the visitor for something. New in A3; the brief requires the form, the event context, validation and a confirmation.
Content:	Left panel: the event being booked - name, date, time, venue, price per ticket and the charity - fetched from the API so the visitor can see what they are booking without going back. Right panel, 'Your details': first and last name, email, contact phone, number of tickets (1-50), date of registration, and optional notes. A live total (tickets x price), and the 'Complete registration' button. After a successful booking the form is replaced by a confirmation panel.
Interactions:	Every field validates as the visitor types and again on submit, each with its own message under the field - no generic alert. Submit calls POST /api/registrations. 201 replaces the form with the confirmation and a link back to the event. 409 shows the API's own sentence: this email has already booked, the event is suspended, or the event has finished. 400 names the exact field to fix. The booking date cannot be in the future, and cannot be later than the event itself - and the ceiling is checked again in the submit handler, because a max attribute on an input is a hint to the browser, not a guarantee to the server.
Media Creation:	None - this page is pure data and form. The total is computed in the component from tickets x the event's price, never stored.
Navigation:	In: the Register button on the event detail page, carrying eventId in the query string. Out: 'Back to the event' link; after booking, back to the event to see your own name in the list.

Storyboard

TITLE: Staff - event list (admin site) SCREEN ID: S5 DATE: 18 October 2026

Elements:

GRAPHICS:



Live screenshot of the deployed page (https://24831941.it.scu.edu.au), taken with headless Chrome. The layout is the design; every value on it comes from the API.

TEXT:

Title: 'Events' and one sentence saying what the site can change
Button: '+ New event'
Filter chips with live counts: All 13 / Active 8 / Past 4 / Suspended 1
Column headings: ID / Event / Date / Location / Status / Ticket / Registrations / Actions
Row buttons: 'Edit' / 'Delete'

AUDIO/VIDEO:

None. Status badges are coloured text, and the counts are rendered from the rows actually on screen.

NOTES

NOTES:

Counts live on the filter chips because 'how many suspended?' is the first question staff ask. A blocked delete reports the reason and the alternative (suspend it) rather than failing silently.

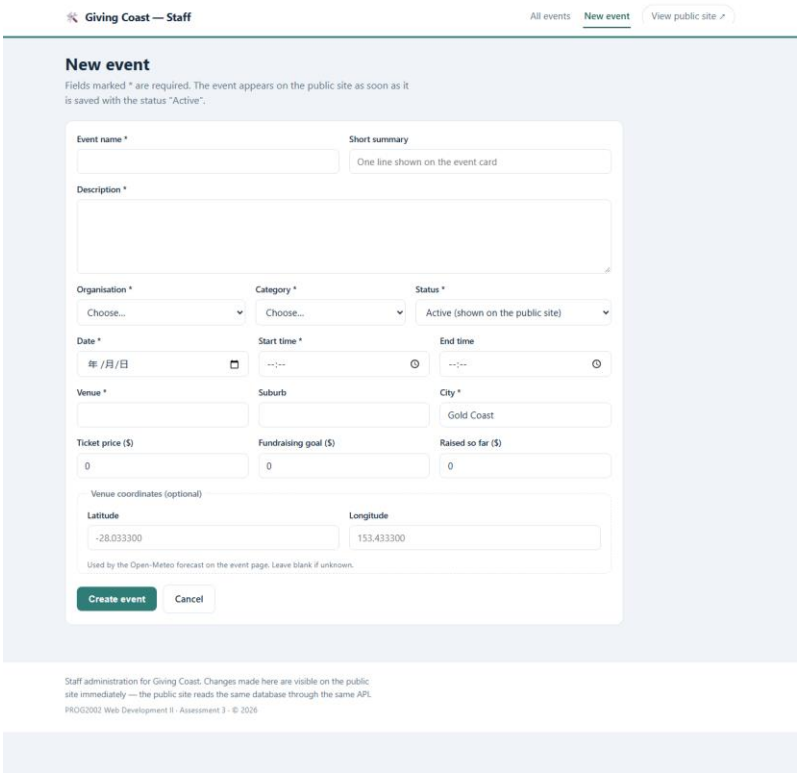
Description:	The staff home page, deployed under /admin/. Every event whatever its status, with the doors to the other two staff screens. Authentication is out of scope by design - the brief says so.
Content:	Title and a one-sentence explanation of what the site can change. '+ New event' button. Status filter chips with live counts - All 13 / Active 8 / Past 4 / Suspended 1 - taken from the rows actually on screen, so a chip can never claim a number the table does not show. The table: ID, Event (with category and charity), Date, Location, Status badge, Ticket price, Registrations (bookings and tickets sold), and Actions. Edit and Delete buttons on every row.
Interactions:	The chips filter the rendered list; suspended wins over past, so a cancelled event stays visible with the reason it was cancelled. Delete asks for confirmation, then calls DELETE /api/events/:id. If bookings exist the API answers 409 and the screen says exactly why instead of failing silently. Edit opens the edit screen for that event; '+ New event' opens the create screen; 'View public site' opens the public site in a new tab.
Media Creation:	Status badges are coloured per status (active, past, suspended). No images.
Navigation:	In: 'Staff admin' in the public menu, or All events in the staff menu. Out: New event, Edit, the public site.

Storyboard

TITLE: Staff - new event SCREEN ID: S6 DATE: 18 October 2026

Elements:

GRAPHICS:



Live screenshot of the deployed page (https://24831941.it.scu.edu.au), taken with headless Chrome. The layout is the design; every value on it comes from the API.

TEXT:

Title: 'New event'
Required fields marked '*': name, description, organisation, category, status, date, start time, venue, city
Optional: summary, end time, suburb, ticket price, goal, raised so far, latitude / longitude ('Used by the Open-Meteo forecast on the event page. Leave blank if unknown.')
Buttons: 'Save changes' / 'Cancel'

AUDIO/VIDEO:

None. The two selects are filled from GET /api/organisations and GET /api/categories, the same lists the public site shows.

NOTES

NOTES:

Create and edit share one form component so the two field sets cannot drift apart; the coordinates are grouped separately and labelled with what they are for.

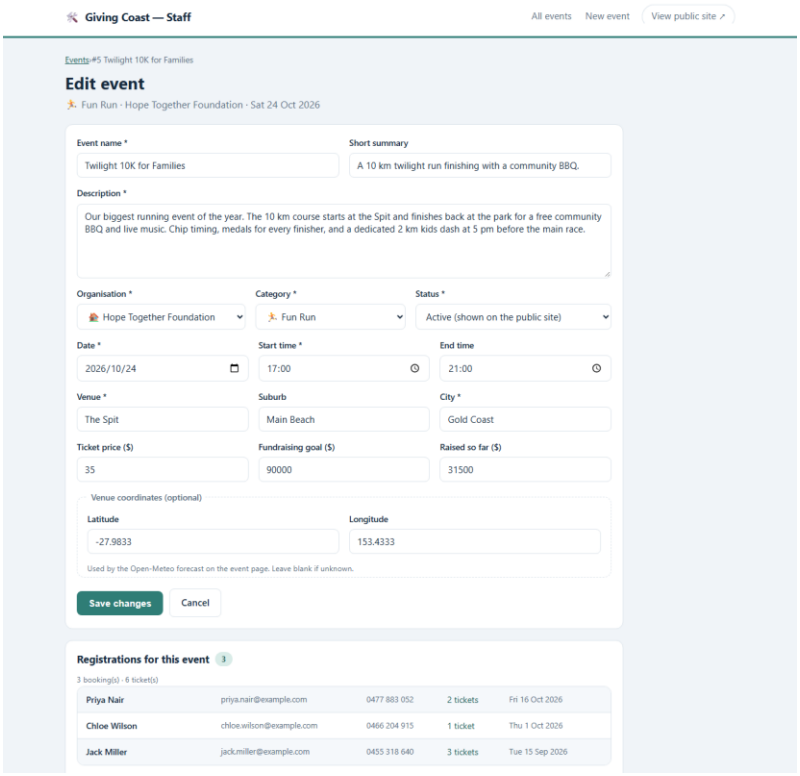
Description:	Create an event. The same form component the edit screen uses, empty - which is why the two screens cannot drift apart.
Content:	Required fields: event name, description, organisation, category, status, date, start time, venue and city. Optional: short summary, end time, suburb, ticket price, fundraising goal, raised so far, and the venue's latitude and longitude in a separate 'Venue coordinates' group with the note 'Used by the Open-Meteo forecast on the event page. Leave blank if unknown.' Save changes / Cancel.
Interactions:	The form is a Reactive Form: the Save button stays disabled until every required field is valid, and each field shows its own message. Save calls POST /api/events and returns to the list, where the new event appears with the status it was given. The organisation and category selects are filled from GET /api/organisations and GET /api/categories - the same lists the public site shows, so a staff member can never invent a category that does not exist. Cancel discards the form and goes back without a request.
Media Creation:	None. The coordinates are plain number inputs, and leaving them blank simply means the public page shows no forecast for that event.
Navigation:	In: '+ New event' on the staff list. Out: back to the list, either by saving or by cancelling.

Storyboard

TITLE: Staff - edit event SCREEN ID: S7 DATE: 18 October 2026

Elements:

GRAPHICS:



Live screenshot of the deployed page (https://24831941.it.scu.edu.au), taken with headless Chrome. The layout is the design; every value on it comes from the API.

TEXT:

Same form as S6, pre-filled (screenshot: the Twilight 10K)
Section 'Registrations for this event': booking count, total tickets, then each booking with name, email, phone, tickets and date
The delete rule in the staff member's own screen: 'Because registrations exist, this event cannot be deleted - suspend it instead.'

AUDIO/VIDEO:

None beyond the data itself - the registration list is part of the same API response as the event.

NOTES

NOTES:

Form and registrations on one screen: the staff member sees who a change affects while making it.

Description:	Change an event, and see who has booked it - the two things a staff member needs on one screen, which is why the registrations sit under the form rather than on a page of their own.
Content:	The same form as S6, pre-filled from GET /api/events/:id (in the screenshot, the Twilight 10K: name, summary, description, organisation, category, status, date and times, venue, city, prices, and the stored coordinates). Underneath, 'Registrations for this event': the booking count and total tickets, then each booking with name, email, phone, tickets and the date it was made. And the sentence that explains the delete rule where the staff member is standing: 'Because registrations exist, this event cannot be deleted - suspend it instead.'
Interactions:	Save calls PUT /api/events/:id with the whole form (PUT rather than PATCH because the screen always submits every field). The coordinates saved here are what drive the weather panel on the public event page - the two features meet in these two fields. The registration list comes from the same endpoint that filled the form, so there is no second round trip and no way for the two to disagree.
Media Creation:	None beyond the data itself.
Navigation:	In: Edit on any row of the staff list (the breadcrumb shows Events > event name). Out: Save changes or Cancel both return to the list.